

Федеральное государственное автономное образовательное учреждение  
высшего образования  
«Национальный исследовательский университет  
«Высшая школа экономики»

Московский институт электроники и математики им. А.Н.Тихонова

Направление подготовки  
«10.03.01 Информационная безопасность»  
Образовательная программа «Информационная безопасность»

**О Т Ч Е Т**  
о прохождении  
учебной практики

Студент Каримов К.И.  
(Фамилия И.О.)

БИБ 215  
номер группы

**Руководитель практики студента:**  
ведущий программист МИЭМ

\_\_\_\_\_  
должность и место работы

Башун В.В.

\_\_\_\_\_  
Фамилия И.О.



\_\_\_\_\_  
подпись

**Руководитель практики от НИУ ВШЭ:**  
Зав. каф. информационной безопасности  
киберфизических систем департамента электронной  
инженерии МИЭМ НИУ ВШЭ

\_\_\_\_\_  
должность и место работы

Евсютин О.О.

\_\_\_\_\_  
Фамилия И.О.

\_\_\_\_\_  
подпись

Практика пройдена с оценкой отлично (8)

Дата 23.02.2023

Москва, 2023

## СОДЕРЖАНИЕ

|                                                                    |    |
|--------------------------------------------------------------------|----|
| 1 Введение                                                         | 3  |
| Алгоритмы:                                                         | 3  |
| 1. Линейная регрессия                                              | 4  |
| 2. Логистическая регрессия                                         | 4  |
| 3. Деревья решений                                                 | 4  |
| 4. Случайный лес                                                   | 5  |
| 5. Искусственная нейронная сеть                                    | 5  |
| 6. Свёрточная нейронная сеть                                       | 5  |
| 7. Рекуррентные нейронные сети / RNN                               | 6  |
| 7.1 Сети LSTM                                                      | 7  |
| Виды компьютерных атак:                                            | 8  |
| 1. Инъекция кода                                                   | 9  |
| 2. Отказ в обслуживании                                            | 10 |
| 3. Червь                                                           | 11 |
| 4. Сетевая разведка                                                | 11 |
| 5. Атака через посредника / Man in the middle / Человек посередине | 12 |
| 6. Сниффинг пакетов                                                | 12 |
| 7. IP-спуфинг                                                      | 12 |
| 8. Brute-force / Полный перебор / Метод «грубой силы»              | 13 |
| 2 Краткая характеристика организации                               | 16 |
| 3 Описание профессиональных задач                                  | 17 |
| 4 Выполненное индивидуальное задание                               | 18 |
| 1. Подготовка датасета                                             | 18 |
| 2. Проверка объектов векторов                                      | 20 |
| 3. Подготовка и вывод имеющихся данных                             | 20 |
| 4. Подготовка нейронной сети                                       | 21 |
| 5. Оценка модели                                                   | 26 |
| 6. Сравнение с моделью случайного леса                             | 27 |
| 5 Заключение                                                       | 32 |
| 6 Список использованных источников                                 | 33 |

## **1 Введение**

Целью прохождения практики является закрепление, расширение и углубление полученных теоретических знаний и приобретение первоначальных практических навыков в решении конкретных проблем.

Задачи практики:

- закрепление и углубление теоретических знаний по прослушанным за время обучения в университете дисциплинам;
- формирование и совершенствование базовых профессиональных навыков и умений в области информационной безопасности;
- знакомство и отработка навыков работы с реальными исследовательскими, промышленными и образовательными проектами;
- формирование информационной компетентности с целью успешной работы в профессиональной деятельности;
- получение навыков самостоятельной работы, а также работы в составе коллектива;
- обработка полученных материалов и оформление отчета о прохождении практики.

**Информационная безопасность (ИБ) / Кибербезопасность** — практика предотвращения несанкционированных действий с информацией. К ним относятся доступ, использование, копирование, раскрытие, искажение, исследование, уничтожение. ИБ включает в себя набор технологий, стандартов и методов управления.

Задачи Кибербезопасности – обеспечение конфиденциальности, целостности и доступности данных. Только комплексный и системный подход позволит обеспечить информационную безопасность. Он должен учитывать все вероятные и существующие угрозы и уязвимости. Контроль за ИБ должен проводиться в круглосуточном режиме и охватывать все этапы существования информации от момента ее появления и до уничтожения либо потери актуальности.

**Машинное обучение / Machine learning / ML** — класс методов искусственного интеллекта, характерной чертой которых является не прямое решение задачи, а обучение за счёт применения решений множества сходных задач. Для построения таких методов используются средства математической статистики, численных методов, математического анализа, методов оптимизации, теории вероятностей, теории графов, различные техники работы с данными в цифровой форме.

Различают два типа обучения:

1. Обучение по прецедентам, или индуктивное обучение, основано на выявлении эмпирических закономерностей в данных.
2. Дедуктивное обучение предполагает формализацию знаний экспертов и их перенос в компьютер в виде базы знаний. Дедуктивное обучение принято относить к области экспертных систем, поэтому термины машинное обучение и обучение по прецедентам можно считать синонимами.

Многие методы индуктивного обучения разрабатывались как альтернатива классическим статистическим подходам. Многие методы тесно связаны с извлечением информации и интеллектуальным анализом данных.

### **Алгоритмы:**

Существует множество алгоритмов машинного обучения, которые применяются на практике. Продемонстрирую самые известные алгоритмы, а также подробно опишу тех, которыегодились мне в процессе прохождения практики, а именно CNN и подвид RNN — LSTM (двунаправленной).

## **1. Линейная регрессия**

Линейная регрессия изначально использовалась в статистике для исследования взаимосвязи между переменными, затем была адаптирована для прогнозирования на основе уравнений линейной регрессии.

## **2. Логистическая регрессия**

Используя логистическую регрессию, мы получаем дискретные значения (1 или 0). Это соответствует ответу да-нет модели на текущие входные данные. Модель логистической регрессии вычисляет взвешенную сумму входных переменных, похожую на линейную регрессию, но она пропускает результат через специальную нелинейную функцию, логистическую функцию или сигмовидную функцию для получения ответа да-нет.

## **3. Деревья решений**

Деревья решений — это древовидный инструмент, который можно использовать для представления причины-следствия. Поскольку одна причина может иметь несколько следствий, мы перечисляем их ниже (как дерево с его ветвями).

Мы можем построить дерево решений, организовав входные данные и переменные этой модели-предсказателя в соответствии с некоторыми критериями.

### **Основные этапы построения дерева решений:**

- a. Получить рыночные данные для финансового инструмента
- b. Синтезировать входные данные (то есть технические индикаторы, индикаторы настроения, индикаторы ширины и так далее)
- c. Для всех входных данных установить целевые значения
- d. Разделить данные на тренировочные и тестовые
- e. Создать дерево решений для обучения модели
- f. Тестировать и анализировать модель

Недостаток деревьев решений следующий, а именно склонность к переобучению из-за их структуры.

#### 4. Случайный лес

Алгоритм случайного леса позволяет обойти ограничения деревьев решений.

Случайный лес состоит из ансамбля деревьев решений, которые представляют собой графы решений с их действиями или статистической вероятностью.

Предсказания деревьев проходят через одно итоговое дерево (CART).

Как работает классификация. Подсчитываются результаты предсказания всех деревьев ансамбля. Выбирается класс, который набрал наибольшее количество голосов.

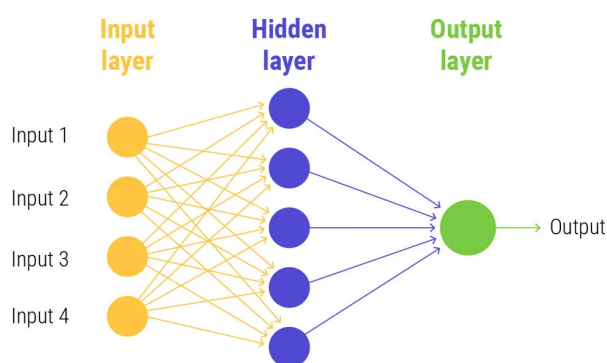
Как работает регрессия. Результаты предсказаний всех деревьев усредняются. Это среднее и является результатом предсказания не отдельного дерева, а всей модели случайного леса.

**Как работает случайный лес:**

- Предположим, что у нас есть  $N$  экземпляров тренировочных данных. Выборка из этих  $N$  экземпляров берется в качестве обучающего набора.
- Рассмотрим  $M$  как число входных признаков. Выбираем другое число  $m$  так, что  $m < M$ . Наилучшее разделение между  $m$  и  $M$  используется для разделения узла. Значение  $m$  не меняется по мере роста деревьев.
- Каждое дерево ансамбля растет по мере обучения.
- Объединяя прогнозы  $n$  деревьев (то есть большинство голосов за классификацию, среднее за регрессию), предсказываются новые данные.

#### 5. Искусственная нейронная сеть

Искусственная нейронная сеть — одно из главных достижений. Нейросеть строится на взаимосвязи узлов между собой — именно так связаны нейроны в нашем мозге. Каждый нейрон получает информацию от другого, выполняет действие и передает его другому нейрону в качестве выходных данных.



Один узел нейронной сети представляет из себя искусственный нейрон, а стрелка обозначает связь между выходом одного нейрона и входом другого.

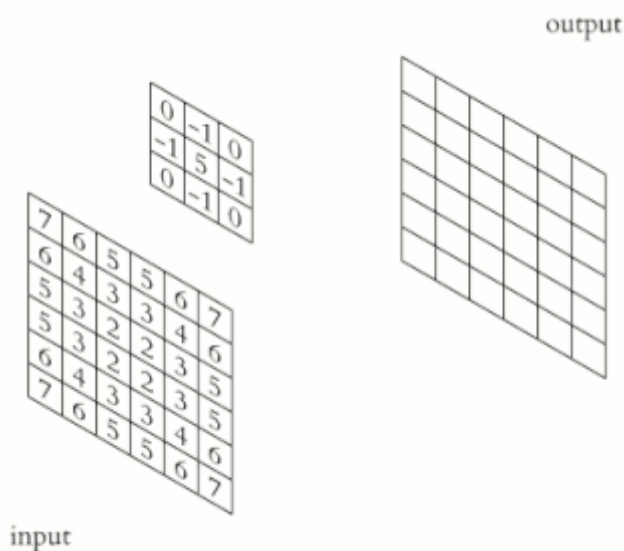
Нейронные сети могут быть более полезными, если используются для нахождения взаимосвязей между различными классами активов.

#### 6. Свёрточная нейронная сеть

Свёрточная нейронная сеть / convolutional neural network / CNN — специальная архитектура искусственных нейронных сетей, предложенная Яном Лекуном в 1988 году и нацеленная на эффективное распознавание образов, входит в состав

технологий глубокого обучения (deep learning). Использует некоторые особенности зрительной коры, в которой были открыты так называемые простые клетки, реагирующие на прямые линии под разными углами, и сложные клетки, реакция которых связана с активацией определённого набора простых клеток. Таким образом, идея свёрточных нейронных сетей заключается в чередовании свёрточных слоёв (convolution layers) и субдискретизирующих слоёв (subsampling layers) / слоёв подвыборки (pooling layers). Структура сети — однонаправленная (без обратных связей), принципиально многослойная. Для обучения используются стандартные методы, чаще всего метод обратного распространения ошибки. Функция активации нейронов (передаточная функция) — любая, по выбору исследователя. Название архитектура сети получила из-за наличия операции свёртки, суть которой в том, что каждый фрагмент изображения умножается на матрицу (ядро) свёртки поэлементно, а результат суммируется и записывается в аналогичную позицию выходного изображения.

3D-анимация хода свёртки:

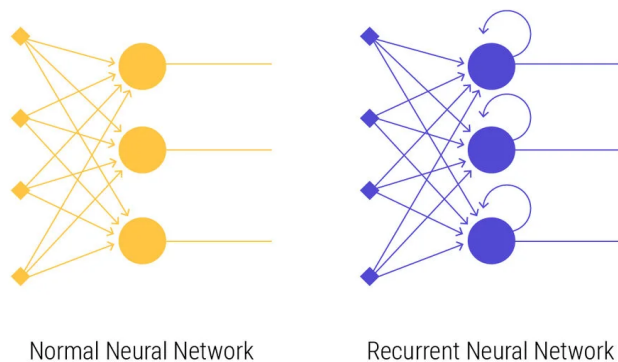


Значения в ядре можно извлечь независимо друг от друга. Для периферийных областей входов применяются различные подходы заполнения.

## 7. Рекуррентные нейронные сети / RNN

Этот алгоритм — тип нейронной сети, в которой к каждому узлу прикреплена память, что упрощает обработку последовательных данных, то есть выходные данные зависят не только от текущих входных данных, но и от входных данных, которые уже были обработаны ранее.

Преимущество рекуррентной нейронной сети над обычной состоит в том, что нам нужно обрабатывать слово символ за символом. Возьмем слово «торговать»: узел нейронной сети забудет символ «т» к тому времени, когда он переместится на «г», тогда как рекуррентная нейронная сеть запомнит этот символ, поскольку у него есть собственная память.

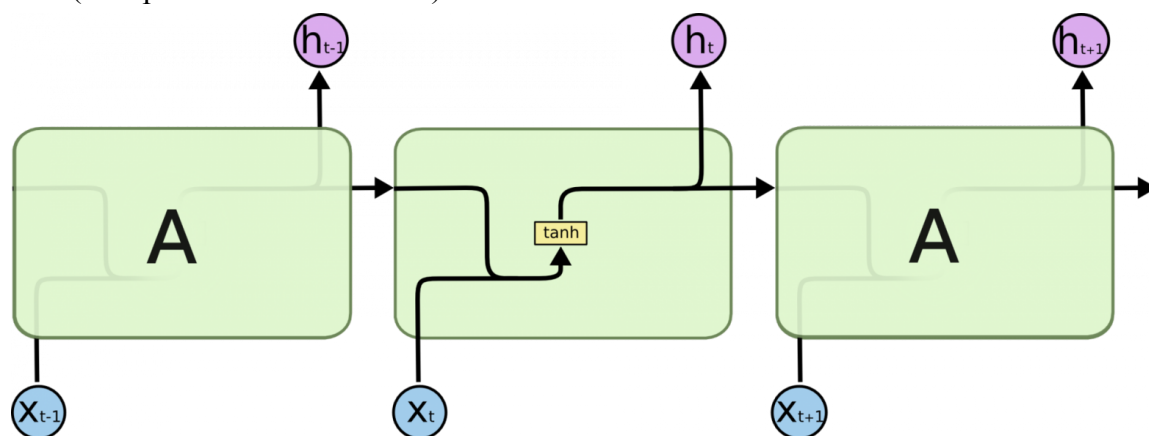


## Сети LSTM

Долгая краткосрочная память (Long short-term memory; LSTM) — особая разновидность архитектуры рекуррентных нейронных сетей, способная к обучению долговременным зависимостям. Они были представлены Зеппом Хохрайтер и Юргеном Шмидхубером (Jürgen Schmidhuber) в 1997 году, а затем усовершенствованы и популярно изложены в работах многих других исследователей. Они прекрасно решают целый ряд разнообразных задач и в настоящее время широко используются.

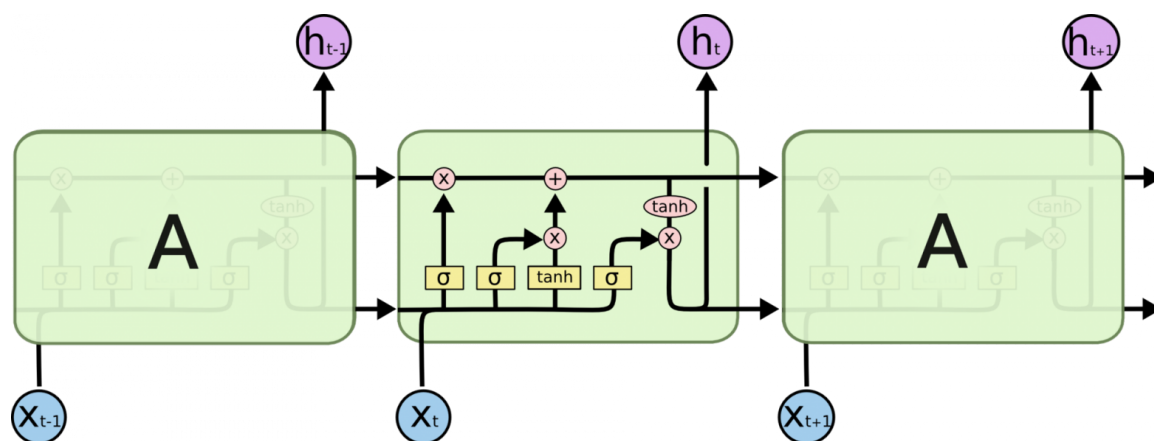
LSTM разработаны специально, чтобы избежать проблемы долговременной зависимости. Запоминание информации на долгие периоды времени — это их обычное поведение, а не что-то, чему они с трудом пытаются обучиться.

Любая рекуррентная нейронная сеть имеет форму цепочки повторяющихся модулей нейронной сети. В обычной RNN структура одного такого модуля очень проста, например, он может представлять собой один слой с функцией активации  $\tanh$  (гиперболический тангенс).



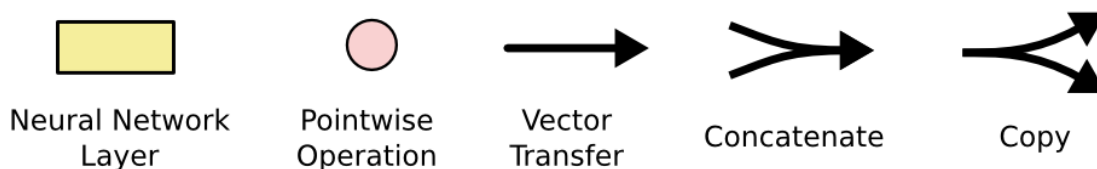
**Повторяющийся модуль в стандартной RNN состоит из одного слоя.**

Структура LSTM также напоминает цепочку, но модули выглядят иначе. Вместо одного слоя нейронной сети они содержат целых четыре, и эти слои взаимодействуют особым образом.



**Повторяющийся модель в LSTM сети состоит из четырех взаимодействующих слоев.**

Не будем пока озадачиваться подробностями. Рассмотрим каждый шаг схемы LSTM позже. Пока познакомимся со специальными обозначениями, которыми мы будем пользоваться.



**Слой нейронной сети; поточечная операция; векторный перенос; объединение; копирование.**

На схеме выше каждая линия переносит целый вектор от выхода одного узла ко входу другого. Розовыми кружочками обозначены поточечные операции, такие, как сложение векторов, а желтые прямоугольники – это обученные слои нейронной сети. Сливающиеся линии означают объединение, а разветвляющиеся стрелки говорят о том, что данные копируются и копии уходят в разные компоненты сети.

### Компьютерные атаки:

Принято считать, что первые компьютерные атаки появились в конце 1960-х или начале 1970-х годов, когда начали взламывать телефонные сети. В мейнфреймах впервые начали внедрять механизмы разделения доступа, чтобы пользователи не мешали друг другу, а также не могли украсть или данные испортить друг друга.

**Взлом этих систем защиты — это компьютерная атака**, которая решает следующую задачу: как обойти все системы защиты, которые разделяют пользователей, и получить такой результат, как будто никаких препятствий нет. Сейчас есть два основных способа таких атак: через технику и через людей.

**Компьютерные атаки** проходят через технику, а **социальная инженерия** — через людей, однако всего, что связано с ней, например фишинга, претекстинга и прочих видов интернет-мошенничества мы касаться не будем.

Поскольку основная тема практики — детектирование **Веб-атак**, то акцентировать внимание будем в первую очередь на них. Выделим их основные виды:

## 1. Инъекция кода

Семейство атак, объединенных одним общим принципом — в результате атаки данные выполняются как код. Данный раздел включает в себя несколько пунктов, поэтому для удобства разобью его на блоки:

**1.1. Внедрение SQL-кода / SQL injection / SQLi** — один из распространённых способов взлома сайтов и программ, работающих с базами данных, основанный на внедрении в запрос произвольного SQL-кода. Внедрение SQL, в зависимости от типа используемой СУБД и условий внедрения, может дать возможность атакующему выполнить произвольный запрос к базе данных (например, прочитать содержимое любых таблиц, удалить, изменить или добавить данные), получить возможность чтения и/или записи локальных файлов и выполнения произвольных команд на атакуемом сервере.

Атака типа внедрения SQL может быть возможна из-за некорректной обработки входных данных, используемых в SQL-запросах.

Разработчик прикладных программ, работающих с базами данных, должен знать о таких уязвимостях и принимать меры противодействия внедрению SQL.

**Существует три основных класса атак, основанных на внедрении SQL-кода:**

- a. Классическая SQL инъекция (Classic SQLi)
- b. SQL инъекция, основанная на эксплуатации выводимых СУБД сообщений об ошибках (Error-based SQLi)
- c. Слепая SQL инъекция (Blind SQLi)

**1.2. PHP-инъекция** — один из способов взлома веб-сайтов, работающих на PHP. Он заключается в том, чтобы внедрить специально сформированный злонамеренный сценарий в код веб-приложения на серверной стороне сайта, что приводит к выполнению произвольных команд. Известно, что во многих распространённых в интернете бесплатных движках и форумах, работающих на PHP (чаще всего это устаревшие версии) есть непродуманные модули или отдельные конструкции с уязвимостями. Крэкеры анализируют такие уязвимости как незэкранированные переменные, получающие внешние значения.

Потенциально опасными функциями являются:

eval(),  
preg\_replace() (с модификатором «e»),  
require\_once(),  
include\_once(),  
include(),  
require(),  
create\_function().

РНР-инъекция становится возможной, если входные параметры принимаются и используются без проверки.

- 1.3. Межсайтовый скриптинг / XSS / Cross-Site Scripting** — тип атаки на веб-системы, заключающийся во внедрении в выдаваемую веб-системой страницу вредоносного кода (который будет выполнен на компьютере пользователя при открытии им этой страницы) и взаимодействии этого кода с веб-сервером злоумышленника. Специфика подобных атак заключается в том, что вредоносный код может использовать авторизацию пользователя в веб-системе для получения к ней расширенного доступа или для получения авторизационных данных пользователя. Вредоносный код может быть вставлен в страницу как через уязвимость в веб-сервере, так и через уязвимость на компьютере пользователя. XSS находится на третьем месте в рейтинге ключевых рисков Web-приложений, согласно OWASP 2021. Долгое время программисты не уделяли им должного внимания, считая их неопасными. Однако это мнение ошибочно: на странице или в HTTP-Cookie могут быть весьма уязвимые данные (например, идентификатор сессии администратора или номера платежных документов), а там, где нет защиты от CSRF, атакующий может выполнить любые действия, доступные пользователю. Межсайтовый скриптинг может быть использован для проведения DoS-атаки, о которой мы поговорим далее.
- 1.4. XPath-инъекция** — вид уязвимостей, который заключается во внедрении XPath-выражений в оригинальный запрос к базе данных XML. Как и при остальных видах инъекций, уязвимость возможна ввиду недостаточной проверки входных данных.
- 1.5. Автозалив** — веб-инъекция, действующая по принципу троянских программ, основная цель которой заключается во внедрении в аккаунт пользователя в платежной системе, незаметной подмене данных транзакции путём модификации HTML-кода и переводе средств пользователя на счёт злоумышленника.

## **2. Отказ в обслуживании**

Атаки типа «отказ в обслуживании» (DoS, DDoS) перегружают системы, серверы, сайты трафиком для нарушения или прекращения их обслуживания, чтобы сделать его недоступным для посетителей. В результате перегруженного сервера система не может отвечать на запросы пользователей и прекращает свою работу.

- 2.1. DoS / Denial of Service / Отказ в обслуживании** — атака, имеющая своей целью заставить сервер не отвечать на запросы. Такой вид атаки не подразумевает получение некоторой секретной информации, но иногда бывает подспорьем в инициализации других атак. Например, некоторые программы из-за ошибок в своём коде могут вызывать исключительные ситуации и при отключении сервисов

способны исполнять код, предоставленный злоумышленником, или атаки лавинного типа, когда сервер не может обработать огромное количество входящих пакетов.

**2.2. DDoS / Distributed Denial of Service / Распределенная DoS** — подтип DoS атаки, имеющий ту же цель, что и DoS, но производимой не с одного компьютера, а с нескольких компьютеров в сети. В данных типах атак используется либо возникновение ошибок, приводящих к отказу сервиса, либо срабатывание защиты, приводящей к блокированию работы сервиса, а в результате также к отказу в обслуживании. DDoS используется там, где обычный DoS неэффективен. Для этого несколько компьютеров объединяются, и каждый производит DoS-атаку на систему жертвы. Вместе это называется DDoS-атака. Размер ботнета может составлять от нескольких десятков до нескольких сотен тысяч компьютеров.

Любая атака представляет собой не что иное, как попытку использовать несовершенство системы безопасности жертвы либо для получения информации, либо для нанесения вреда системе, поэтому причиной любой удачной атаки является профессионализм крэкера и ценность информации, а также недостаточная компетенция администратора системы безопасности, в частности, несовершенство программного обеспечения и недостаточное внимание к вопросам безопасности в компании в целом.

### **3. Червь**

Сетевой червь — тип вредоносных программ, распространяющихся по сетевым каналам, способных к автономному преодолению систем защиты автоматизированных и компьютерных систем, а также к созданию и дальнейшему распространению своих копий, не всегда совпадающих с оригиналом, осуществлению иного вредоносного воздействия.

### **4. Сетевая разведка**

Сетевая разведка — получение и обработка данных об информационной системе клиента, ресурсов информационной системы, используемых устройств и программного обеспечения и их уязвимостях, средств защиты, а также о границе проникновения в информационную систему.

Сетевая разведка проводится в форме запросов DNS, эхо-тестирования (ping) и сканирования портов. Запросы DNS помогают понять, кто владеет тем или иным доменом и какие адреса этому домену присвоены. Эхо-тестирование адресов, раскрытых с помощью DNS, позволяет увидеть, какие хосты реально работают в данной среде. Получив список хостов, хакер использует средства сканирования портов, чтобы составить полный список услуг, поддерживаемых этими хостами. И, наконец, хакер анализирует характеристики приложений, работающих на хостах. В результате добывается информация, которую можно использовать для взлома.

## **5. Атака через посредника / Man in the middle / Человек посередине**

MITM — тип атаки, при котором киберпреступник становится «третьим лишним» и пропускает весь веб-трафик через себя. В этот момент потенциальная жертва ни о чем не подозревает, что приводит к тому, что все учетные данные для входа в системы оказываются у злоумышленника. После полученная информация может быть использована для кражи корпоративных данных или несанкционированных переводов средств. Одним из примеров атак типа «человек посередине» является активное прослушивание, при котором злоумышленник устанавливает независимые связи с жертвами и передает сообщения между ними. Тем самым он заставляет жертв поверить, что они разговаривают непосредственно друг с другом через частную связь, фактически же весь разговор управляется злоумышленником. Злоумышленник должен уметь перехватывать все передаваемые между двумя жертвами сообщения, а также вводить новые. В большинстве случаев это довольно просто: например, злоумышленник может вести себя как «человек посередине» в пределах диапазона приёма беспроводной точки доступа (Wi-Fi)

## **6. Сниффинг пакетов**

Также довольно распространенный вид атаки, основанный на работе сетевой карты в режиме promiscuous mode, а также monitor mode для сетей Wi-Fi. В таком режиме все пакеты, полученные сетевой картой, пересылаются на обработку специальному приложению, называемому сниффером. В результате злоумышленник может получить большое количество служебной информации: кто, откуда и куда передавал пакеты, через какие адреса эти пакеты проходили. Самой большой опасностью такой атаки является получение самой информации, например логинов и паролей сотрудников, которые можно использовать для незаконного проникновения в систему под видом обычного сотрудника компании.

## **7. IP-спуфинг**

Это вид хакерской атаки, заключающийся в использовании чужого IP-адреса источника с целью обмана системы безопасности.

Метод, используемый в некоторых атаках. Состоит в изменении поля «адрес отправителя» IP-пакета. Применяется с целью сокрытия истинного адреса атакующего, с целью вызвать ответный пакет на нужный адрес, а также с иными целями.

Простейший способ проверить, что подозрительный пакет пришел от верного отправителя — отправить пакет на IP отправителя. Обычно для IP-спуфинга используется случайный IP, и вполне вероятно, что ответ не придёт. Если же придёт, имеет смысл сравнить поле TTL (Time to live) полученных пакетов. Если поля не совпадают — пакеты пришли из разных источников. На сетевом уровне атака частично предотвращается с помощью фильтра пакетов на шлюзе. Он должен быть настроен таким образом, чтобы

не пропускать пакеты, пришедшие через те сетевые интерфейсы, откуда они прийти не могли. Например, фильтрация пакетов из внешней сети с исходным адресом внутри сети. Одним из самых надёжных методов защиты от подмены IP-адреса является сопоставление MAC-адреса (Ethernet кадр) и IP-адреса (заголовок протокола IP) отправителя. Например, если пакет с IP адресом из внутренней сети имеет MAC адрес шлюза — этот пакет стоит отбросить. В современных сетевых устройствах изменение MAC-адреса (физического адреса) не является проблемным.

## **8. Brute-force / Полный перебор / Метод «грубой силы»**

Brute-force — метод решения математических задач. Относится к классу методов поиска решения исчерпыванием всевозможных вариантов. Сложность полного перебора зависит от количества всех возможных решений задачи. Если пространство решений очень велико, то полный перебор может не дать результатов в течение нескольких лет или даже столетий.

Любая задача из класса NP может быть решена полным перебором. При этом, даже если вычисление целевой функции от каждого конкретного возможного решения задачи может быть осуществлено за полиномиальное время, в зависимости от количества всех возможных решений полный перебор может потребовать экспоненциального времени работы.

В криптографии на вычислительной сложности полного перебора основывается оценка криптостойкости шифров. Шифр считается криптостойким, если не существует метода «взлома» существенно более быстрого чем полный перебор всех ключей. Криптографические атаки, основанные на методе полного перебора, являются самыми универсальными, но и самыми долгими.

Криптографические атаки, основанные на методе «грубой силы», являются наиболее универсальными, но в то же время наиболее медленными. Используются в основном начинающими хакерами. Эффективны для несложных алгоритмов шифрования и ключей длиной до 64 бит. Для современных ключей длиной от 128 бит (иногда для ключа факторизируется большое число из 200 цифр), неэффективны. Любой пароль может быть подобран путём полного перебора.

В следующей таблице представлено оценочное время полного перебора паролей в зависимости от их длины. Предполагается, что в пароле могут использоваться 36 различных символов (латинские буквы одного регистра + цифры), а скорость перебора составляет 100 000 паролей в секунду.

| Кол-во знаков | Кол-во вариантов               | Стойкость | Время перебора |
|---------------|--------------------------------|-----------|----------------|
| 1             | 36                             | 5 бит     | менее секунды  |
| 2             | 1296                           | 10 бит    | менее секунды  |
| 3             | 46 656                         | 15 бит    | менее секунды  |
| 4             | 1 679 616                      | 21 бит    | 17 секунд      |
| 5             | 60 466 176                     | 26 бит    | 10 минут       |
| 6             | 2 176 782 336                  | 31 бит    | 6 часов        |
| 7             | 78 364 164 096                 | 36 бит    | 9 дней         |
| 8             | $2,821\,109\,9 \times 10^{12}$ | 41 бит    | 11 месяцев     |
| 9             | $1,015\,599\,5 \times 10^{14}$ | 46 бит    | 32 года        |
| 10            | $3,656\,158\,4 \times 10^{15}$ | 52 бита   | 1 162 года     |
| 11            | $1,316\,217\,0 \times 10^{17}$ | 58 бит    | 41 823 года    |
| 12            | $4,738\,381\,3 \times 10^{18}$ | 62 бита   | 1 505 615 лет  |

Таким образом, пароли длиной до 8 символов включительно в общем случае не являются надежными. Для современных компьютеров этот показатель гораздо выше. Так, 64-битный ключ (пароль) перебирается на современном компьютере примерно за 2 года и перебор легко может быть распределен между множеством компьютеров.

### **Устойчивость к атаке полного перебора**

В процессе улучшения системы информационной безопасности по отношению к атаке полным перебором можно выделить два основных направления:

- повышение требований к ключам доступа от защищаемой информации;
- повышение надежности всех узлов системы безопасности.

Таким образом, невозможно достичь высокого уровня защиты, улучшая только один из этих параметров. Существуют примеры того, как система аутентификации, основанная на оптимальной сложности паролей, оказывалась уязвимой для копирования базы данных на локальный компьютер злоумышленника, после чего подвергалась brute force атаке с применением локальных оптимизаций и вычислительных средств, недоступных при удаленном криптоанализе. Из этого можно сделать вывод, что длинные пароли гораздо более криптоустойчивы. Ниже приведен список некоторых применяемых на практике методов повышения надежности криптосистемы по отношению к brute force атаке:

- Защита базы данных от копирования.
- Хеширование паролей на веб-сервере.

- Применение корректно настроенного межсетевого экрана, например, iptables.
- Другие средства, препятствующие быстрой проверке корректности ключа, например, Captcha, а также запрет или тайм-аут аутентификации пользователя.

## **2 Краткая характеристика организации**

Место прохождения практики — Национальный Исследовательский Институт “Высшая Школа Экономики”. Факультет — “Московский институт электроники и математики”. Образовательная программа — “Информационная безопасность”.

### **3 Описание профессиональных задач**

Профессиональными задачами, решаемыми на практике, является получение навыков по применению алгоритмов машинного обучения для детектирования компьютерных атак и веб-фаерволов, а именно углубленное изучение современных веб-атак и технологий глубинного и машинного обучения для детектирования угроз.

#### **Применение алгоритмов на практике**

Алгоритмы машинного обучения могут помочь в решении задач ИБ, вовремя обнаруживая угрозу и устраняя её. В теории, обученная на множестве датасетов, нейросеть может предотвращать как известные атаки, раскрытие конфиденциальной информации и прочее, так и атаку нулевого дня, когда те или иные уязвимости не устранены. Нейросети стремительно набирают обороты и постоянно самосовершенствуются, однако им всё ещё нужны время и ресурсы, которые позволить себе может далеко не каждая компания. Однако находятся энтузиасты, разработки которых могут подтолкнуть прогресс развития ML в сфере ИБ.

#### 4 Выполненное индивидуальное задание

В процессе выполнения практики мною была исследована модель глубинного обучения CNN-BiLSTM для обнаружения сетевых вторжений. Модель основывается на сверточных нейронных сетях (CNN), о которых я рассказывал в пункте алгоритмы и разновидности LSTM, а именно — Bi-LSTM.

Что она из себя представляет? Двусторонний или двунаправленный (BiLSTM / Bidirectional LSTM) LSTM — это тип сети LSTM, в которой обучающие данные подаются от начала до конца модели, одновременно передавая информацию от конца к началу, что позволяет улучшить обучение на каждом временном шаге данных. Это приводит к лучшему изучению функций за каждый временной шаг. BiLSTM состоит из двух сетей LSTM: одна — принимает входные данные в прямом направлении, а другая — в обратном. BiLSTM эффективно увеличивает объем информации, доступной для сети, улучшая контекст, доступный для алгоритма (например, зная, какие слова следуют непосредственно за словом в предложении и предшествуют ему).

Более подробно об алгоритме работы модели будет рассказано в пункте **подготовка нейронной сети**.

##### 1. Подготовка датасета

Начнём с самих данных. Датасет (далее дс) — является упрощенным и отфильтрованным, основанном на дс от Канадского института Кибербезопасности. В него входит безобидный трафик, который будет помечен как BENIGN (безобидный), brute-force атаки, sql-инъекции, DoS/DDoS и прочие виды веб-атак. Оригинальный датасет содержит данные за 5 дней, однако их общий объем превышает десятки гигабайт, что будет излишне для нашего эксперимента. Обойдемся упрощенной версией от автора реализации. В датасете имеется 7267 строк на 76 колонок с продолжительностью потока, общей длиной пакетов, количество возвращенных пакетов и так далее. Подробнее о наполнении датасета будет упомянуто в дальнейшем.

Для работы алгоритмы понадобится произвести импорт следующих библиотек:

```
✓ 7 [1] from itertools import cycle
    import pickle
    import sys
    import time
    from typing import List, Tuple

    import matplotlib.pyplot as plt
    import numpy as np
    import pandas as pd
    from sklearn import metrics
    from sklearn.model_selection import StratifiedKFold
    from sklearn.model_selection import train_test_split
    import sklearn.preprocessing
    import tensorflow as tf
    import tensorflow.keras
    from tensorflow.keras.layers import Dense, Dropout, Activation
    from tensorflow.keras.layers import LSTM, Bidirectional, BatchNormalization, Convolution1D, MaxPooling1D, Reshape
    from tensorflow.keras.models import Sequential

    %matplotlib inline
```

Основными, конечно, являются tensorflow с модулями нормализации, набивки и прочего, а также sklearn.

После успешной загрузки датасета:

```
✓ 1 !wget https://github.com/fisher85/ml-cybersecurity/blob/master/python-web-attack-detection/datasets/web_attacks_balanced.zip?raw=true -O dataset.zip
    !unzip -u dataset.zip

--2023-03-22 11:56:25-- https://github.com/fisher85/ml-cybersecurity/blob/master/python-web-attack-detection/datasets/web_attacks_balanced.zip?raw=true
Resolving github.com (github.com)... 192.30.255.112
Connecting to github.com (github.com)|192.30.255.112|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://github.com/fisher85/ml-cybersecurity/raw/master/python-web-attack-detection/datasets/web_attacks_balanced.zip [following]
--2023-03-22 11:56:25-- https://github.com/fisher85/ml-cybersecurity/raw/master/python-web-attack-detection/datasets/web_attacks_balanced.zip
Reusing existing connection to github.com:443.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/fisher85/ml-cybersecurity/master/python-web-attack-detection/datasets/web_attacks_balanced.zip [following]
--2023-03-22 11:56:26-- https://raw.githubusercontent.com/fisher85/ml-cybersecurity/master/python-web-attack-detection/datasets/web_attacks_balanced.zip
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.109.133, 185.199.111.133, 185.199.108.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.109.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 695583 (679K) [application/zip]
Saving to: 'dataset.zip'

dataset.zip      100%[=====] 679.28K  --.-KB/s   in 0.03s

2023-03-22 11:56:26 (21.6 MB/s) - 'dataset.zip' saved [695583/695583]

Archive: dataset.zip
  inflating: web_attacks_balanced.csv
```

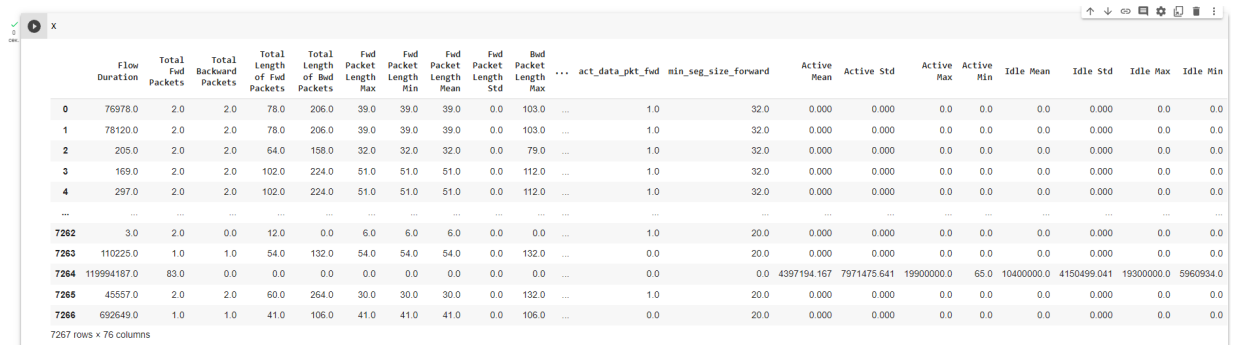
Необходимо убрать 7 объектов, а именно идентификатор потока, IP-адрес источника, порт источника, IP-адрес назначения, порт назначения, протокол и временная метка. Делается это потому что "форма" передаваемых данных более важна, чем эти атрибуты. Кроме того, порты и адреса могут быть подменены злоумышленником, поэтому лучше, чтобы алгоритм ML не учитывал эти особенности при обучении.

```
✓ 0 data = pd.read_csv('web_attacks_balanced.csv', delimiter=',')

    y = data['Label']
    X = data.drop(labels=['Label', 'Flow ID', 'Source IP', 'Source Port',
        'Destination IP', 'Destination Port', 'Protocol', 'Timestamp'], axis=1)
```

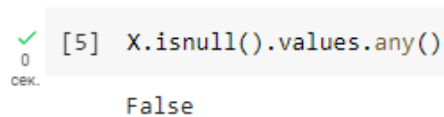
## 2. Проверка объектов векторов

Выведем наш датасет X:



|      | Flow Duration | Total Fwd Packets | Total Backward Packets | Total Length of Fwd Packets | Total Length of Bud Packets | Fwd Packet Length Max | Fwd Packet Length Min | Fwd Packet Length Mean | Fwd Packet Length Std | Bud Packet Length Max | ... | act_data_pkt_fwd | min_seg_size_forward | Active Mean | Active Std  | Active Max | Active Min | Idle Mean  | Idle Std    | Idle Max   | Idle Min  |
|------|---------------|-------------------|------------------------|-----------------------------|-----------------------------|-----------------------|-----------------------|------------------------|-----------------------|-----------------------|-----|------------------|----------------------|-------------|-------------|------------|------------|------------|-------------|------------|-----------|
| 0    | 76978.0       | 2.0               | 2.0                    | 78.0                        | 206.0                       | 39.0                  | 39.0                  | 39.0                   | 0.0                   | 103.0                 | ... | 1.0              | 32.0                 | 0.000       | 0.000       | 0.0        | 0.0        | 0.0        | 0.000       | 0.0        | 0.0       |
| 1    | 78120.0       | 2.0               | 2.0                    | 78.0                        | 206.0                       | 39.0                  | 39.0                  | 39.0                   | 0.0                   | 103.0                 | ... | 1.0              | 32.0                 | 0.000       | 0.000       | 0.0        | 0.0        | 0.0        | 0.000       | 0.0        | 0.0       |
| 2    | 206.0         | 2.0               | 2.0                    | 64.0                        | 158.0                       | 32.0                  | 32.0                  | 32.0                   | 0.0                   | 79.0                  | ... | 1.0              | 32.0                 | 0.000       | 0.000       | 0.0        | 0.0        | 0.0        | 0.000       | 0.0        | 0.0       |
| 3    | 169.0         | 2.0               | 2.0                    | 102.0                       | 224.0                       | 51.0                  | 51.0                  | 51.0                   | 0.0                   | 112.0                 | ... | 1.0              | 32.0                 | 0.000       | 0.000       | 0.0        | 0.0        | 0.0        | 0.000       | 0.0        | 0.0       |
| 4    | 297.0         | 2.0               | 2.0                    | 102.0                       | 224.0                       | 51.0                  | 51.0                  | 51.0                   | 0.0                   | 112.0                 | ... | 1.0              | 32.0                 | 0.000       | 0.000       | 0.0        | 0.0        | 0.0        | 0.000       | 0.0        | 0.0       |
| ...  | ...           | ...               | ...                    | ...                         | ...                         | ...                   | ...                   | ...                    | ...                   | ...                   | ... | ...              | ...                  | ...         | ...         | ...        | ...        | ...        | ...         | ...        | ...       |
| 7262 | 3.0           | 2.0               | 0.0                    | 12.0                        | 0.0                         | 6.0                   | 6.0                   | 6.0                    | 0.0                   | 0.0                   | ... | 1.0              | 20.0                 | 0.000       | 0.000       | 0.0        | 0.0        | 0.0        | 0.000       | 0.0        | 0.0       |
| 7263 | 110225.0      | 1.0               | 1.0                    | 54.0                        | 132.0                       | 54.0                  | 54.0                  | 54.0                   | 0.0                   | 132.0                 | ... | 0.0              | 20.0                 | 0.000       | 0.000       | 0.0        | 0.0        | 0.0        | 0.000       | 0.0        | 0.0       |
| 7264 | 119994187.0   | 83.0              | 0.0                    | 0.0                         | 0.0                         | 0.0                   | 0.0                   | 0.0                    | 0.0                   | 0.0                   | ... | 0.0              | 0.0                  | 4397194.167 | 7971475.641 | 19900000.0 | 65.0       | 10400000.0 | 4150499.041 | 19300000.0 | 5960994.0 |
| 7265 | 45557.0       | 2.0               | 2.0                    | 60.0                        | 264.0                       | 30.0                  | 30.0                  | 30.0                   | 0.0                   | 132.0                 | ... | 1.0              | 20.0                 | 0.000       | 0.000       | 0.0        | 0.0        | 0.0        | 0.000       | 0.0        | 0.0       |
| 7266 | 692649.0      | 1.0               | 1.0                    | 41.0                        | 106.0                       | 41.0                  | 41.0                  | 41.0                   | 0.0                   | 106.0                 | ... | 0.0              | 20.0                 | 0.000       | 0.000       | 0.0        | 0.0        | 0.0        | 0.000       | 0.0        | 0.0       |

И проверим наличие пустых элементов:

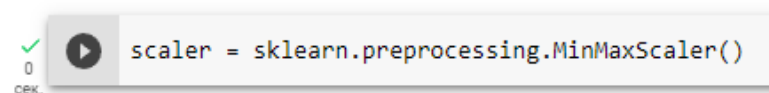


```
[5] X.isnull().values.any()
```

False

, как видим, NULL элементы отсутствуют.

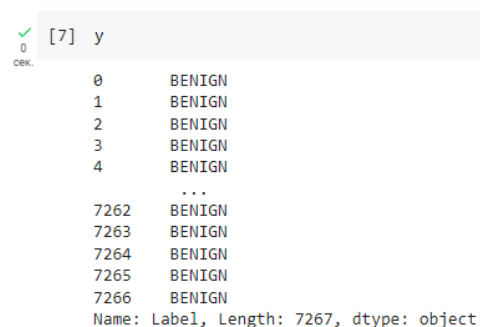
Диапазон функций здесь не нормализован. Нам необходимо выполнить масштабирование объектов для правильного функционирования выбранных алгоритмов глубокого обучения (CNN и BiLSTM). В нашем случае используется нормализация min-max: все выбранные объекты преобразуются в диапазон [0, 1].



```
scaler = sklearn.preprocessing.MinMaxScaler()
```

Чтобы предотвратить утечку данных, масштабирование объектов будет выполняться при перекрестной проверке сгибов на этапе обучения.

## 3. Подготовка и вывод имеющихся данных



```
[7] y
```

0 BENIGN  
1 BENIGN  
2 BENIGN  
3 BENIGN  
4 BENIGN  
...  
7262 BENIGN  
7263 BENIGN  
7264 BENIGN  
7265 BENIGN  
7266 BENIGN  
Name: Label, Length: 7267, dtype: object

Также, как и в прошлый раз, проверим дс на наличие пустых значений:

```
✓ 0 сек. y.isnull().values.any()  
False
```

Как мы можем видеть, ярлыки категоричны. Нам нужно было бы преобразовать их в числовую форму перед обучением нейронной сети. Чтобы достичь этого, мы бы применили к меткам одноразовое кодирование.

Чтобы предотвратить утечку данных, однократное кодирование будет выполняться в рамках перекрестной проверки на этапе обучения.

#### 4. Подготовка нейронной сети

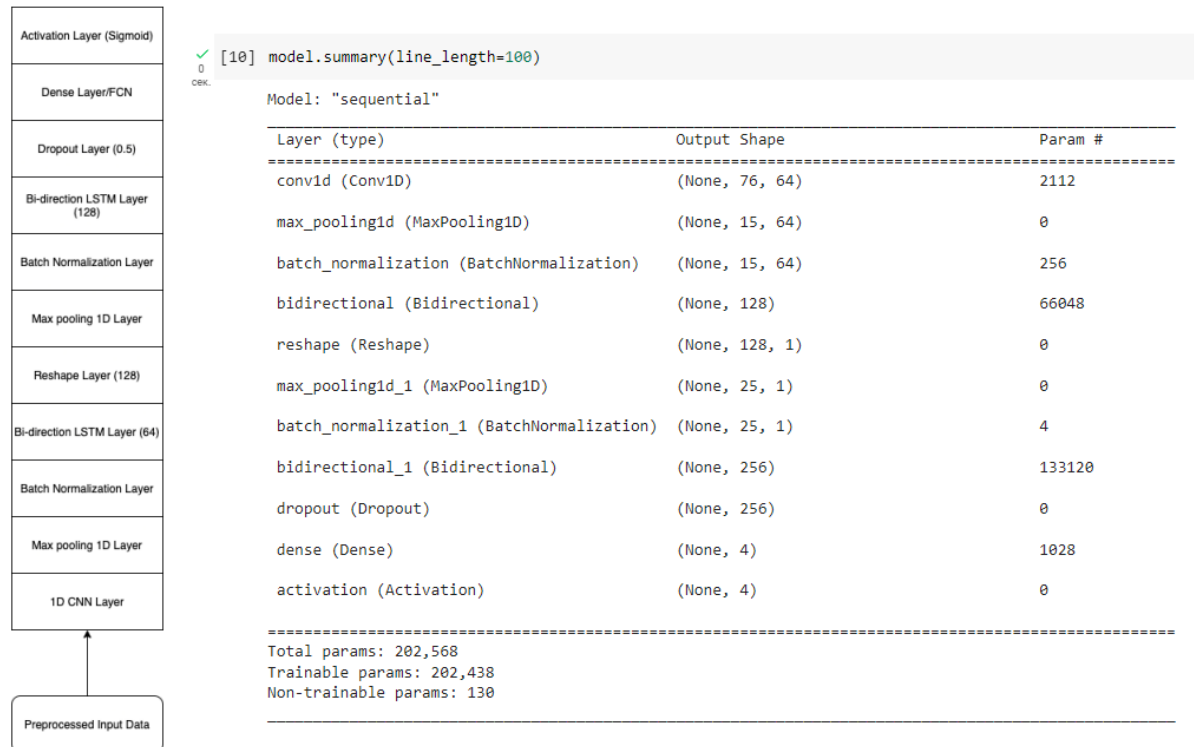
Скомпилируем нашу сеть. В работе мы используем последовательную модель со слоями CNN и BiLSTM, а пакетная нормализация применяется перед слоями BiLSTM. Отсевной слой наносится перед последним, чтобы предотвратить чрезмерную подгонку.

Категориальная перекрестная энтропия выбрана в качестве функции потерь из-за многоклассовой классификации. Чтобы получить результат классификации, функция активации softmax применяется к выходным данным последнего, полностью подключенного слоя.

```
✓ 2 сек. [9] batch_size = 32  
model = Sequential()  
model.add(Convolution1D(64, kernel_size=32, padding="same",  
                        activation="relu", input_shape=(76, 1)))  
model.add(MaxPooling1D(pool_size=(5)))  
model.add(BatchNormalization())  
model.add(Bidirectional(LSTM(64, return_sequences=False)))  
model.add(Reshape((128, 1), input_shape=(128, )))  
  
model.add(MaxPooling1D(pool_size=(5)))  
model.add(BatchNormalization())  
model.add(Bidirectional(LSTM(128, return_sequences=False)))  
  
model.add(Dropout(0.5))  
model.add(Dense(4))  
model.add(Activation('softmax'))  
model.compile(loss='categorical_crossentropy',  
              optimizer='adam', metrics=['accuracy'])
```

Чтобы лучше понять алгоритм нейросети, я проведу более глубокий экскурс в суть её работы.

Перед нами блочная диаграмма, описывающая пошаговую работу модели (слева) и её интерпретация в формате кода (справа):



Как видно из блок-схемы, предлагаемая модель состоит из одномерного слоя CNN вместе с несколькими слоями Bi-LSTM со слоями Reshape и Batch Normalization между ними. Идея состоит в том, чтобы использовать 1-D слой (одномерный CNN) и слой max pooling для совместного использования параметров, пространственного расположения и особенности местного восприятия. Совместное использование параметров позволяет сократить набор параметров и свободных переменных, что приводит к извлечению объектов с меньшим использованием ресурсов обработки.

Пространственное расположение позволяет расположить в разреженной матрице характеристик признаки для обеспечения лучшего распознавания корреляции между ними. Наконец, местное восприятие позволяет уменьшить количество параметров и, следовательно, уменьшает продолжительность обучения в разы. Из этого можно сделать вывод, что 1-D CNN обеспечивает быстрое пространственное обучение для данного временного ряда данных. За одномерным слоем следует max pooling, позволяющий производить выборочную дискретизацию параметров, чтобы

распознавать соответствующие функции, что приводит к сокращению времени обучения и предотвращению переобучения. После max pooling, появляется слой пакетной нормализации, который включает нормализацию параметров между промежуточными слоями для предотвращения более медленного времени обучения. Изменение формы слоев после пакетной нормализации слоев изменяет выходные данные предыдущего слоя для следующей пары слоев Bi-LSTM. Слои Bi-LSTM используются для обучения как по прямым, так и по обратным данным временных рядов, при этом скрытые слои используют два блока, имеющие один и тот же вход и подключенные к одному и тому же выходу. Один из модулей обрабатывает прямые временные ряды и другие процессы обратного временного ряда. Это называется договоренностью, которая обеспечивает слои будущими данными для увеличения времени обучения за счет лучшего изучения функций в результате с большей точностью для длинных временных рядов данных.

Два слоя Bi-LSTM в модели расположены с помощью способа, который удваивает размер ядра на каждой итерации. Согласно блок-схеме модели, первый слой Bi-LSTM начинается с 64 единиц, а последующий и последний слой Bi-LSTM имеет 128 единиц. Причиной этого выбора является имитация использования крупнозернистой модели, чтобы лучше понять корреляцию дальнего действия, зависящую от времени функции, изученные одномерным CNN, который обеспечивает лучшее извлечение функций и более быстрое время обучения. Между каждым уровнем Bi-LSTM имеется слой max pooling для удаления наименее релевантных функций, а также слои пакетной нормализации для нормализации выходных данных предыдущего промежуточного уровня с целью повышения производительности и сокращения времени обучения.

Перейдём непосредственно к подготовке и тренировке нашей сети. В нашем эксперименте мы используем стратифицированную перекрестную проверку K-кратности, которая позволяет оценить производительность некоторой модели в наборе данных. Работает это так, что набор данных разделяется случайным образом на k групп, одна из них выбирается и таким образом рассчитывается среднеквадратичная ошибка. Далее процесс повторяется k раз, используя каждый раз другой набор в качестве набора удержания. Общая среднеквадратичная ошибка считается как среднее значение k тестовых MSE:

```
✓ [11] cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
0
сек.
```

Оценочные показатели сохраняются для каждого сгиба:

```
✓ [12] scores = pd.DataFrame(columns=[1, 2, 3, 4, 5, 'Mean'],
0
сек.                               index=['Accuracy', 'Precision', 'Recall', 'F1'])
```

Для получения оценочных показателей, а именно accuracy, precision, recall и f1 мы используем следующую функцию:

```
✓ [13] def print_metrics(y_eval: np.ndarray, y_pred: np.ndarray, average: str = 'binary') -> List[float]:
0
сек.     accuracy = metrics.accuracy_score(y_eval, y_pred)
        precision = metrics.precision_score(y_eval, y_pred, average=average)
        recall = metrics.recall_score(y_eval, y_pred, average=average)
        f1 = metrics.f1_score(y_eval, y_pred, average=average)

        print('Accuracy =', accuracy)
        print('Precision =', precision)
        print('Recall =', recall)
        print('F1 =', f1)

        return [accuracy, precision, recall, f1]
```

Классическая матрица ошибок (confusion matrix) выглядит следующим образом:

|               | $y = 1$             | $y = 0$             |
|---------------|---------------------|---------------------|
| $\hat{y} = 1$ | True Positive (TP)  | False Positive (FP) |
| $\hat{y} = 0$ | False Negative (FN) | True Negative (TN)  |

Используется она для оценки качества моделей и сравнения различных алгоритмов

Все тестовые и прогнозируемые метки также сохраняются на каждом этапе перекрестной проверки. Они будут использоваться для вычисления матрицы ошибок:

```
✓ [14] actual_targets = np.empty([0], dtype=float)
0
сек.     predicted_targets = np.empty([0], dtype=float)
```

При каждом сгибе мы используем следующую функцию для преобразования данных, подгонки и проверки нейронной сети. Обратим внимание, что предварительная обработка данных в основном выполняется на этапе перекрестной проверки, чтобы предотвратить утечку данных:

```

0 сек.
def train_and_validate(model: Sequential,
                       X: pd.DataFrame,
                       y: pd.Series,
                       train_index: np.ndarray,
                       test_index: np.ndarray,
                       scaler: sklearn.preprocessing.MinMaxScaler) -> Tuple[np.ndarray, np.ndarray]:
    # Получаем тестовые данные и данные проверки для текущего сгиба:
    X_train, X_test = X.iloc[train_index], X.iloc[test_index]
    y_train, y_test = y[train_index], y[test_index]

    print("Train index:", train_index)
    print("Test index:", test_index)

    # Проводим масштабирование данных
    X_train = scaler.fit_transform(X_train)
    X_test = scaler.fit_transform(X_test)

    # Изменяем форму векторов объектов, чтобы использовать их в качестве входных данных нейронной сети.
    X_train = np.reshape(
        X_train, (X_train.shape[0], X_train.shape[1], 1))
    X_test = np.reshape(
        X_test, (X_test.shape[0], X_test.shape[1], 1))

    # Одноразовое кодирование для меток (названий).
    y_train_encoded = pd.get_dummies(y_train).values
    y_test_encoded = pd.get_dummies(y_test).values

    # Обучение (тренировка) модели на обучающей выборке.
    model.fit(X_train, y_train_encoded, validation_data=(
        X_test, y_test_encoded), epochs=10)

    # Применяем модель нейронной сети к данным проверки и получаем точность этого прогноза.
    y_pred = model.predict(X_test)
    y_pred = np.argmax(y_pred, axis=1)
    y_eval = np.argmax(y_test_encoded, axis=1)

    return y_eval, y_pred

```

Шаг обучения предназначен для использования с различными версиями tensorflow, если это необходимо. Также можно выбрать, должно ли обучение выполняться CPU или GPU.

```

15 мин.
[18] # Tensorflow 2.X, производим запуск на обычном процессоре
with tf.device('/CPU:0'):

    # Tensorflow 2.X, при желании можно произвести запуск и на графическом
    # with tf.device('/GPU:0'):

    # Установим таймер для оценки времени обучения..
    start_time = time.time()

    i = 1
    # Стратифицированная перекрестная проверка K-кратности.
    for train_index, test_index in cv.split(X, y):
        y_eval, y_pred = train_and_validate(
            model, X, y, train_index, test_index, scaler)
        # Сохраняем тестовые и прогнозируемые этикетки - labels
        actual_targets = np.append(actual_targets, y_eval)
        predicted_targets = np.append(predicted_targets, y_pred)
        # Получаем и распечатываем результаты проверки
        print("Validation scores:")
        scores[i] = print_metrics(y_eval, y_pred, average='weighted')
        i += 1

    # Get the learning time.
    print('Total operation time: ', time.time() - start_time, 'seconds')

```

## 5. Оценка модели

Спустя некоторое время — в моем случае 15 минут, модель обучилась. Теперь мы можем произвести оценку модели. В следующем блоке кода мы можем увидеть оценочные показатели для каждого сгиба и их средние значения:

✓  
0  
сек.

[19] scores['Mean'] = scores.mean(axis=1)  
scores

|                  | 1        | 2        | 3        | 4        | 5        | Mean     |
|------------------|----------|----------|----------|----------|----------|----------|
| <b>Accuracy</b>  | 0.892710 | 0.881018 | 0.699931 | 0.896765 | 0.699931 | 0.814071 |
| <b>Precision</b> | 0.917825 | 0.813043 | 0.489904 | 0.892768 | 0.489904 | 0.720689 |
| <b>Recall</b>    | 0.892710 | 0.881018 | 0.699931 | 0.896765 | 0.699931 | 0.814071 |
| <b>F1</b>        | 0.861807 | 0.842975 | 0.576381 | 0.862510 | 0.576381 | 0.744011 |

Визуализируем матрицу ошибок для всех сгибов данных, чтобы оценить производительность классификатора. Здесь и далее мы используем следующую функцию:

✓  
0  
сек.

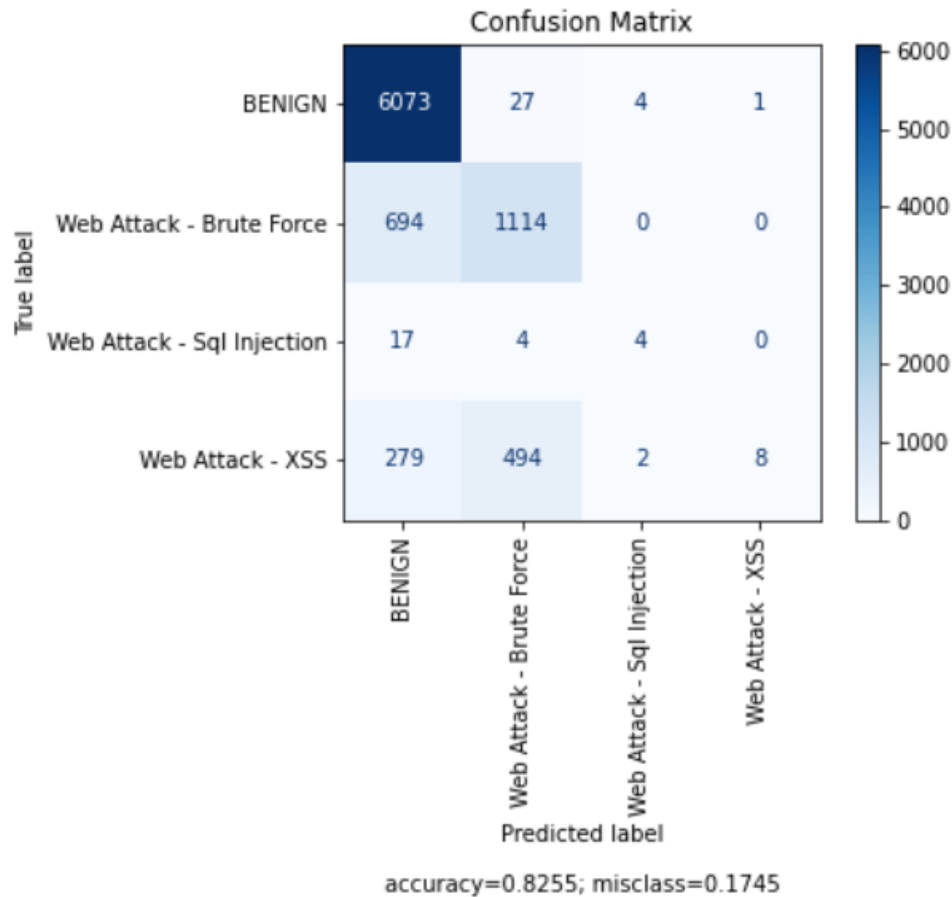
[20] def plot\_confusion\_matrix(y\_eval: np.ndarray, y\_pred: np.ndarray, labels: List[str]) -> None:  
# Получаем матрицу ошибок для последнего сгиба данных:  
confusion\_matrix = metrics.confusion\_matrix(y\_eval, y\_pred)  
  
# Оцениваем показатели точности и неправильной классификации:  
accuracy = np.trace(confusion\_matrix) / float(np.sum(confusion\_matrix))  
misclass = 1 - accuracy  
  
# Построим саму матрицу:  
metrics.ConfusionMatrixDisplay.from\_predictions(  
y\_eval, y\_pred, display\_labels=labels,  
xticks\_rotation='vertical', cmap='Blues')  
plt.xlabel('Predicted label\n\naccuracy={:0.4f}; misclass={:0.4f}'.format(  
accuracy, misclass))  
plt.title('Confusion Matrix')  
plt.show()

Выведем на матрице наши labels по оси x и y. По оси x будут располагаться предсказанные значения, а по y — действительные. Отсортируем их в следующем порядке:

✓  
1  
сек.

[21] plot\_confusion\_matrix(actual\_targets, predicted\_targets, [  
'BENIGN', 'Web Attack - Brute Force', 'Web Attack - Sql Injection', 'Web Attack - XSS'])

Итоговая матрица выглядит следующим образом:



Из увиденного, можем сделать вывод, что лучше всего модель справляется с идентификацией безобидного трафика — 6073 пакета и brute-force атаками — 1114 пакетов. С SQL-инъекциями и XSS модель справилась хуже — 4 и 8 пакетов, соответственно.

## 6. Сравнение с моделью случайного леса

Скачаем и откроем похожую модель для этого датасета, но использующую другой алгоритм, о котором я упоминал ранее — случайный лес.

```
[22] wget https://github.com/fisher85/ml-cybersecurity/blob/master/python-web-attack-detection/webattack_detection_rf_model.pkl?raw=true -O webattack_detection_rf_model.pkl

--2023-03-22 18:09:09-- https://github.com/fisher85/ml-cybersecurity/blob/master/python-web-attack-detection/webattack_detection_rf_model.pkl?raw=true
Resolving github.com (github.com)... 140.82.113.4
Connecting to github.com (github.com)|140.82.113.4|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/fisher85/ml-cybersecurity/master/python-web-attack-detection/webattack_detection_rf_model.pkl [following]
--2023-03-22 18:09:09-- https://raw.githubusercontent.com/fisher85/ml-cybersecurity/master/python-web-attack-detection/webattack_detection_rf_model.pkl
Reusing existing connection to github.com:443.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/fisher85/ml-cybersecurity/master/python-web-attack-detection/webattack_detection_rf_model.pkl [following]
--2023-03-22 18:09:10-- https://raw.githubusercontent.com/fisher85/ml-cybersecurity/master/python-web-attack-detection/webattack_detection_rf_model.pkl
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 547891 (535K) [application/octet-stream]
Saving to: 'webattack_detection_rf_model.pkl'

webattack_detection 100%[=====] 535.05K --.-KB/s in 0.006s

2023-03-22 18:09:10 (82.8 MB/s) - 'webattack_detection_rf_model.pkl' saved [547891/547891]
```

```

✓ [23] with open('webattack_detection_rf_model.pkl', 'rb') as f:
0      rfc = pickle.load(f)
CEK.      rfc

/usr/local/lib/python3.9/dist-packages/sklearn/base.py:318: UserWarning: Try
https://scikit-learn.org/stable/model\_persistence.html#security-maintainability
warnings.warn(
/usr/local/lib/python3.9/dist-packages/sklearn/base.py:318: UserWarning: Try
https://scikit-learn.org/stable/model\_persistence.html#security-maintainability
warnings.warn(
RandomForestClassifier
RandomForestClassifier(max_depth=17, max_features=10, min_samples_leaf=3,
n_estimators=50, oob_score=True, random_state=42)

```

Мы используем индексы конечного сгиба стратифицированной перекрестной проверки K-кратности, чтобы разделить набор данных на обучающий набор с 80% всех выборок и тестовый набор с 20% всех выборок, сохраняя процент выборок для каждого класса.

Модель случайного леса не нуждается в масштабировании объектов и использует только выбранные объекты, поэтому нам нужно получить правильные векторы объектов.

```

✓ [24] columns = ['Average Packet Size',
0              'Flow Bytes/s',
CEK.              'Max Packet Length',
              'Fwd Packet Length Mean',
              'Fwd IAT Min',
              'Total Length of Fwd Packets',
              'Flow IAT Mean',
              'Fwd Packet Length Max',
              'Fwd IAT Std',
              'Fwd Header Length']
X_test = X.iloc[test_index]
X_test = X_test[columns]
X_test

```

|      | Average Packet Size | Flow Bytes/s | Max Packet Length | Fwd Packet Length Mean | Fwd IAT Min | Total Length of Fwd Packets | Flow IAT Mean | Fwd Packet Length Max | Fwd IAT Std | Fwd Header Length |
|------|---------------------|--------------|-------------------|------------------------|-------------|-----------------------------|---------------|-----------------------|-------------|-------------------|
| 5    | 63.500000           | 1.353659e+06 | 79.0              | 32.000000              | 1.0         | 64.0                        | 54.666667     | 32.0                  | 0.000000    | 64.0              |
| 7    | 8.000000            | 9.000000e+06 | 6.0               | 6.000000               | 1.0         | 18.0                        | 1.000000      | 6.0                   | 0.000000    | 60.0              |
| 8    | 236.750000          | 6.491228e+06 | 207.0             | 207.000000             | 4.0         | 414.0                       | 38.000000     | 207.0                 | 0.000000    | 40.0              |
| 10   | 517.818182          | 1.140000e+07 | 1434.0            | 404.000000             | 1.0         | 2828.0                      | 50.100000     | 1405.0                | 134.077217  | 164.0             |
| 11   | 407.466667          | 6.481442e+06 | 1539.0            | 345.333333             | 1.0         | 3108.0                      | 67.357143     | 1539.0                | 284.760269  | 204.0             |
| ...  | ...                 | ...          | ...               | ...                    | ...         | ...                         | ...           | ...                   | ...         | ...               |
| 7248 | 9.000000            | 1.200364e+01 | 6.0               | 6.000000               | 999697.0    | 12.0                        | 999697.000000 | 6.0                   | 0.000000    | 40.0              |
| 7249 | 9.000000            | 9.917355e+04 | 6.0               | 6.000000               | 121.0       | 12.0                        | 121.000000    | 6.0                   | 0.000000    | 40.0              |
| 7251 | 28.750000           | 3.471698e+05 | 23.0              | 23.000000              | 46.0        | 46.0                        | 88.333333     | 23.0                  | 0.000000    | 64.0              |
| 7253 | 0.000000            | 0.000000e+00 | 0.0               | 0.000000               | 0.0         | 0.0                         | 75.000000     | 0.0                   | 0.000000    | 32.0              |
| 7262 | 9.000000            | 4.000000e+06 | 6.0               | 6.000000               | 3.0         | 12.0                        | 3.000000      | 6.0                   | 0.000000    | 40.0              |

1453 rows x 10 columns

Преобразуем категориальные метки в числовую форму с помощью простого кодирования меток. Здесь необходимо обратить внимание на то, что модель случайного леса была разработана для задачи двоичной классификации, поэтому мы соответствующим образом преобразуем метки: "0" указывает на безопасный экземпляр, а "1" - на атаку:

```

✓ [25] y_test = y[test_index]
0 y_test = y_test.apply(lambda x: 0 if x == 'BENIGN' else 1).values
DBK.

```

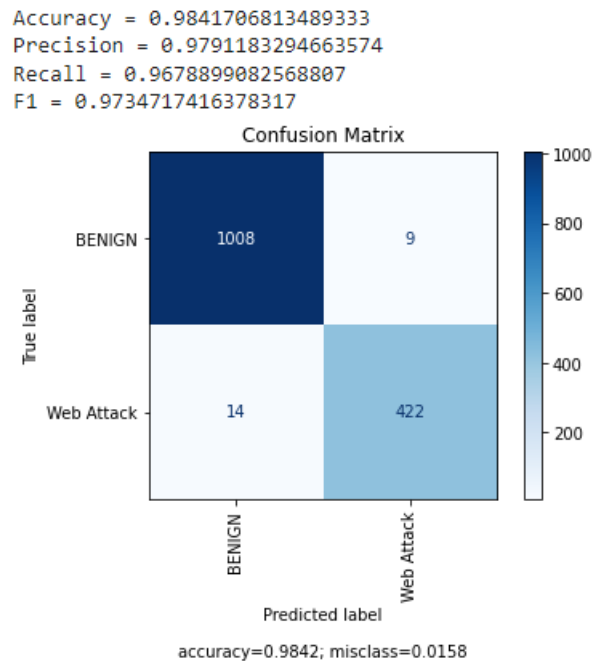
Получаем оценочные показатели модели случайного леса для окончательного сгиба данных:

```

✓ [26] y_pred_rfc = rfc.predict(X_test.values)
1
DBK.

metrics_rfc = print_metrics(y_test, y_pred_rfc)
plot_confusion_matrix(y_test, y_pred_rfc, ['BENIGN', 'Web Attack'])

```



Модель случайного леса предназначена для двоичной классификации, в то время как модель CNN-BiLSTM предназначена для многоклассовой классификации. Чтобы правильно сравнить две модели, мы изменяем форму вывода плотного слоя модели нейронной сети таким образом, чтобы нейронная сеть предсказывала только два класса — безобидный трафик и веб-атаку.

```

✓ [27] batch_size = 32
1
DBK.

model = Sequential()
model.add(Convolution1D(64, kernel_size=32, padding="same",
    activation="relu", input_shape=(76, 1)))
model.add(MaxPooling1D(pool_size=(5)))
model.add(BatchNormalization())
model.add(Bidirectional(LSTM(64, return_sequences=False)))
model.add(Reshape((128, 1), input_shape=(128, )))

model.add(MaxPooling1D(pool_size=(5)))
model.add(BatchNormalization())
model.add(Bidirectional(LSTM(128, return_sequences=False)))

model.add(Dropout(0.5))
model.add(Dense(2))
model.add(Activation('softmax'))
model.compile(loss='categorical_crossentropy',
    optimizer='adam', metrics=['accuracy'])

```

```

✓ [28] model.summary(line_length=100)
0 сек.
Model: "sequential_1"

```

| Layer (type)                               | Output Shape   | Param # |
|--------------------------------------------|----------------|---------|
| conv1d_1 (Conv1D)                          | (None, 76, 64) | 2112    |
| max_pooling1d_2 (MaxPooling1D)             | (None, 15, 64) | 0       |
| batch_normalization_2 (BatchNormalization) | (None, 15, 64) | 256     |
| bidirectional_2 (Bidirectional)            | (None, 128)    | 66048   |
| reshape_1 (Reshape)                        | (None, 128, 1) | 0       |
| max_pooling1d_3 (MaxPooling1D)             | (None, 25, 1)  | 0       |
| batch_normalization_3 (BatchNormalization) | (None, 25, 1)  | 4       |
| bidirectional_3 (Bidirectional)            | (None, 256)    | 133120  |
| dropout_1 (Dropout)                        | (None, 256)    | 0       |
| dense_1 (Dense)                            | (None, 2)      | 514     |
| activation_1 (Activation)                  | (None, 2)      | 0       |

```

Total params: 202,054
Trainable params: 201,924
Non-trainable params: 130

```

Шаги модели те же, что и были в пункте **подготовка нейронной сети**.

Тренируем модифицированную модель на финальном сгиге данных:

```

✓ [30] # Трансформируем Labels в числовые значения:
3 мин.
y = data['Label'].apply(lambda x: 0 if x == 'BENIGN' else 1).values

y_eval, y_pred = train_and_validate(
    model, X, y, train_index, test_index, scaler)

```

Получаем оценку CNN-BiLSTM для бинарной классификации:

```

✓ [31] metrics_nn = print_metrics(y_eval, y_pred)
0 сек.
plot_confusion_matrix(y_eval, y_pred, ['BENIGN', 'Web Attack'])

```

Accuracy = 0.7150722642807984  
 Precision = 1.0  
 Recall = 0.05045871559633028  
 F1 = 0.09606986899563319

Confusion Matrix

|                 | Predicted BENIGN | Predicted Web Attack |
|-----------------|------------------|----------------------|
| True BENIGN     | 1017             | 0                    |
| True Web Attack | 414              | 22                   |

accuracy=0.7151; misclass=0.2849

Проведем сравнение с моделью случайного леса:

```
[74] # Визуализируем данные с помощью столбчатой диаграммы

# Масштабируем
plt.figure(figsize=[15, 9])
plt.gca().set_ylim(0.8, 1)

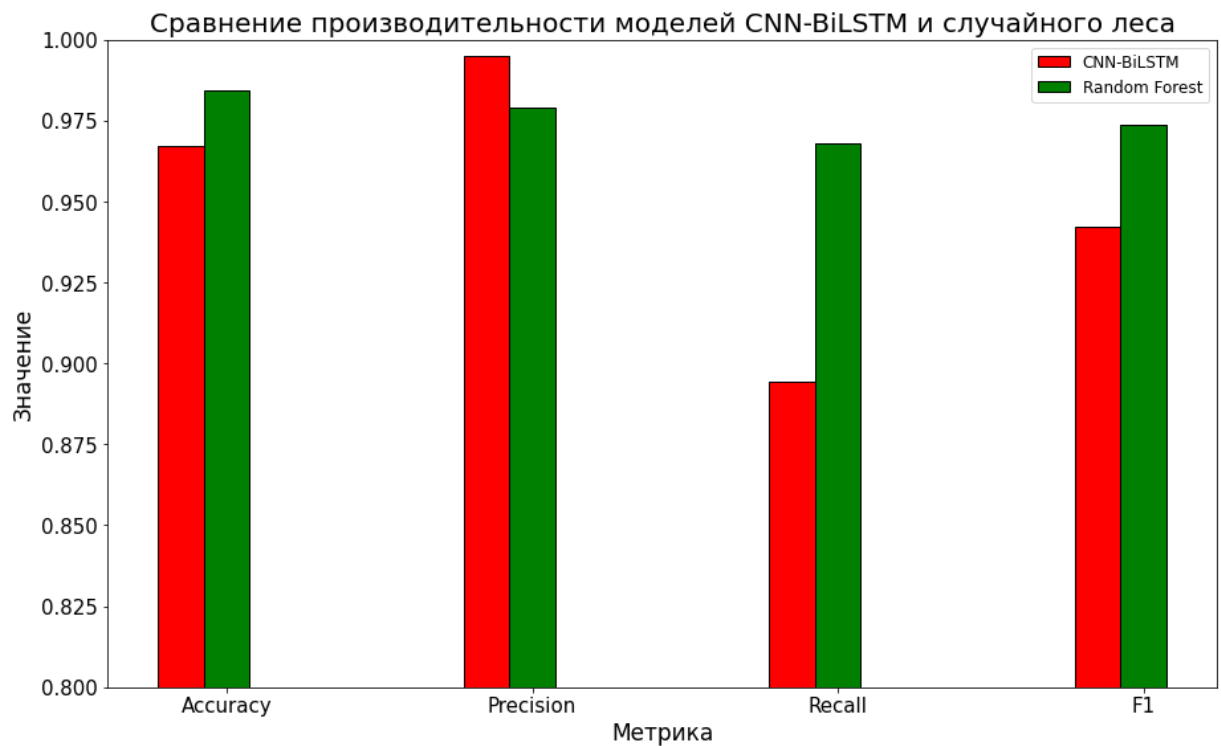
# Получаем данные для графика
comparison_frame = pd.DataFrame(
    {'CNN-BiLSTM': metrics_nn, 'Random Forest': metrics_rfc},
    index=['Accuracy', 'Precision', 'Recall', 'F1'])

# Установим ширину столбцов
wd = 0.3
x_pos = np.arange(1, 2*len(comparison_frame), 2)

# Добавляем несколько диаграмм на один график
plt.bar(x_pos, comparison_frame['CNN-BiLSTM'], color='r', width=wd, edgecolor='k',
        label='CNN-BiLSTM')
plt.bar(x_pos+wd, comparison_frame['Random Forest'], color='g', width=wd, edgecolor='k',
        label='Random Forest')

# Добавим значения по осям X и Y
plt.xticks(x_pos+wd, comparison_frame.index.values, fontsize=15)
plt.yticks(fontsize=15)
plt.xlabel('Метрика', fontsize=17)
plt.ylabel('Значение', fontsize=17)
plt.title(
    "Сравнение производительности моделей CNN-BiLSTM и случайного леса", fontsize=20)
plt.legend(loc='best', fontsize=12)

plt.show()
```



Как можем увидеть, случайный лес превосходит CNN-BiLSTM почти во всех метриках, кроме precision. Тем не менее, CNN-BiLSTM показывает хорошие результаты. Для достижения желаемой производительности необходимы дальнейшие исследования и доработка модели.

## 5 Заключение

В результате прохождения практики мною были исследованы методы применения машинного обучения в детектировании Веб-атак, а также изучена и обучена модель CNN-BiLSTM для детектирования вредоносного трафика. Также я сравнил её с другой моделью, основанной на алгоритме случайного леса на том же датасете и сделал вывод, что обе модели показывают достойные результаты, несмотря на то, что CNN-BiLSTM проигрывает в сравнении со случайным лесом.

Данный эксперимент позволил мне лучше понять, как именно работают свёрточные и рекуррентные нейронные сети, как машинное и глубинное обучение может быть использовано в детектировании вредоносного веб-трафика. Эта информация будет крайне полезной для моего дальнейшего развития в сфере IT, а именно в таких направлениях как Кибербезопасность и Анализ Данных.

Из проделанного я могу сделать вывод, что несмотря на то, что машинное обучение и искусственный интеллект требуют большого развития, методы, применяемые в данной сфере, могут быть использованы во многих сферах деятельности, в том числе и Кибербезопасности, ведь чем больше создаётся систем, тем больше их будут ломать, а самое ключевое отличие ИИ от человека заключается в самом главном — отсутствии человеческого фактора.

## 6 Список использованных источников

1. Efficient Deep CNN-BiLSTM Model for Network Intrusion Detection. URL - [https://jaysinha.me/files/aipr\\_20\\_ids\\_paper\\_pre\\_print.pdf](https://jaysinha.me/files/aipr_20_ids_paper_pre_print.pdf)
2. Bidirectional recurrent neural networks. URL - [https://en.wikipedia.org/wiki/Bidirectional\\_recurrent\\_neural\\_networks](https://en.wikipedia.org/wiki/Bidirectional_recurrent_neural_networks)
3. Метрики в задачах машинного обучения. URL - <https://habr.com/ru/company/ods/blog/328372/>
4. Intrusion Detection Evaluation Dataset (CIC-IDS2017). URL - <https://www.unb.ca/cic/datasets/ids-2017.html>
5. Основные виды атак на инфраструктуру и концепция защиты от них. - URL: <https://onlanta.ru/press/smi/osnovnye-vidy-atak-na-infrastrukturu-i-kontseptsiya-zashchity-ot-nikh/>
6. Машинное обучение: методы и способы. - URL: <https://www.osp.ru/cio/2018/05/13054535>
7. Топ 10 алгоритмов машинного обучения для новичков. - URL: <https://eldf.ru/top10ml>
8. Хакерская Атака. - URL: [https://ru.wikipedia.org/wiki/%D0%A5%D0%B0%D0%BA%D0%B5%D1%80%D1%81%D0%BA%D0%B0%D1%8F\\_%D0%B0%D1%82%D0%B0%D0%BA%D0%B0](https://ru.wikipedia.org/wiki/%D0%A5%D0%B0%D0%BA%D0%B5%D1%80%D1%81%D0%BA%D0%B0%D1%8F_%D0%B0%D1%82%D0%B0%D0%BA%D0%B0)
9. Внедрение SQL-кода. URL - [https://ru.wikipedia.org/wiki/%D0%92%D0%BD%D0%B5%D0%B4%D1%80%D0%B5%D0%BD%D0%B8%D0%B5\\_SQL-%D0%BA%D0%BE%D0%B4%D0%B0](https://ru.wikipedia.org/wiki/%D0%92%D0%BD%D0%B5%D0%B4%D1%80%D0%B5%D0%BD%D0%B8%D0%B5_SQL-%D0%BA%D0%BE%D0%B4%D0%B0)
10. PHP-инъекция. URL - <https://ru.wikipedia.org/wiki/PHP-%D0%B8%D0%BD%D1%8A%D0%B5%D0%BA%D1%86%D0%B8%D1%8F>
11. Межсайтовый скриптинг. URL - [https://ru.wikipedia.org/wiki/%D0%9C%D0%B5%D0%B6%D1%81%D0%B0%D0%B9%D1%82%D0%BE%D0%B2%D1%8B%D0%B9\\_%D1%81%D0%BA%D1%80%D0%B8%D0%BF%D1%82%D0%B8%D0%BD%D0%B3](https://ru.wikipedia.org/wiki/%D0%9C%D0%B5%D0%B6%D1%81%D0%B0%D0%B9%D1%82%D0%BE%D0%B2%D1%8B%D0%B9_%D1%81%D0%BA%D1%80%D0%B8%D0%BF%D1%82%D0%B8%D0%BD%D0%B3)
12. DoS-атака. URL - <https://ru.wikipedia.org/wiki/DoS-%D0%B0%D1%82%D0%B0%D0%BA%D0%B0>
13. Полный перебор. URL - [https://ru.wikipedia.org/wiki/%D0%9F%D0%BE%D0%BB%D0%BD%D1%8B%D0%B9\\_%D0%BF%D0%B5%D1%80%D0%B5%D0%B1%D0%BE%D1%80](https://ru.wikipedia.org/wiki/%D0%9F%D0%BE%D0%BB%D0%BD%D1%8B%D0%B9_%D0%BF%D0%B5%D1%80%D0%B5%D0%B1%D0%BE%D1%80)
14. Простое руководство по K-кратной перекрестной проверке. URL - <https://www.codecamp.ru/blog/k-fold-cross-validation/>
15. Efficient Deep CNN-BiLSTM Model for Network Intrusion Detection. URL - <https://paperswithcode.com/paper/efficient-deep-cnn-bilstm-model-for-network>
16. LSTM – сети долгой краткосрочной памяти. URL - <https://habr.com/ru/company/wunderfund/blog/331310/>
17. Метрики в задачах машинного обучения. URL - <https://habr.com/ru/company/ods/blog/328372/>
18. Детектирование Веб-атак с использованием модели CNN-BiLSTM.ipynb. URL - <https://colab.research.google.com/drive/1IEZoXbeZ1C8nG9r25ZheauQ16imSQpzk?usp=sharing>